```
[Page(UrlRegex = "localhost:8080/#/welcome")]
public class LoginPage : Page
{
  public TextField Email
  {
    get { return Document.TextField(Find.ByName("email")); }
  }
  public TextField Password
  {
    get { return Document.TextField(Find.ByName("password")); }
  }
  public Button SigninButton
  {
    get { return Document.TextField(
                      Find.ByCSSSelector(".btn[value='Sign in']")); }
  }
  public void SignInAs(String userEmail, String userPassword) {
    Email.TypeText(userEmail);
    Password.TypeText(userPassword);
    SignInButton.Click();
  }
}
```

WatiN page objects extend the Page base class.

The UrlRegex is used to check that a test is looking at the right page.

TextFields, Buttons, and so on are used to encapsulate the web elements.

The corresponding test code might look something like this:

```
using (var browser = new IE("http://localhost:8080"))
{
    var loginPage = browser.Page<LoginPage>();
    loginPage.SignInAs("jane@acme.com", "s3cr3t");

    var homePage = browser.Page<HomePage>();
    var message = homePage.WelcomeMessage.Text
    Assert.That(message, Is.EqualTo("Welcome Jane"));
}
```

Set up the browser.

Specify the action under test.

Create the page objects.[5]

Check the test outcomes.

As in the Thucydides code from the previous example, the WatiN page objects neatly hide the web page implementation details from the test logic, making the test code cleaner, more concise, and more readable.

FLUENT SELECTORS

Dynamic languages like Ruby and Groovy make it easy to write fluent and readable APIs, and these can be used to write concise, readable expressions for identifying web elements on a page. Geb provides a powerful expression language based on a jQuery-like notation that allows you to write expressions using CSS selectors. The following code shows a few examples of this sort of expression:

```
$("#search").click()
assert $("#search").disabled
$(".featured-destination").find("a")
$(".featured-destination:nth-child(2) a").click()
```

1 Click on the search button.

2 Verify that the search button is disabled.

3 Find all of the <a> elements inside featured destinations.

4 Click on the link in the second featured destination entry.

[5] For conciseness, we haven't included the implementation of the HomePage page object here.